

Lavi: A Language-Native Self-Growing AI Architecture

Preprint — 2026-06-03

Notation

This paper uses the following terms.

- **Lavi**: The autonomous self-growing AI system proposed by this work.
 - **Loa**: A Claude Code CLI agent that functions as an external evaluator and conversational partner within the experimental environment.
 - **Gles**: The designer and author of Lavi, a user who, together with Loa, serves as an initial evaluator.
-

Abstract

This paper presents an architecture proposal. No empirical experiments are reported. We propose Lavi, a language-native self-growing AI architecture designed to address six structural defects of the Transformer. Lavi comprises four layers: L1, a Mamba2 state-space model providing $O(n)$ sequence processing; L2, LaEMe (Language-based Evaluative Metadata), a general-purpose framework that attaches natural-language metadata to any experience, perception, or interaction, and that subsumes the principle/procedure structure of MARS; L3, Active Inference / DEM, performing free-energy minimization and prediction-error learning; and L4, a Darwin Godel Machine enabling architectural self-improvement. We report the achievement levels for each defect honestly: the $O(n^2)$ complexity problem and the lack of a memory structure are resolved; quasi-fixed weights and the lack of self-explanation are largely mitigated; hallucination is mitigated; and pretraining-data dependence is partially addressed, redefined as post-acquisition continual learning. As a proposal, these claims await empirical validation.

Section 1 : Introduction

Defects of the Transformer (in order of importance)

Defect 1 : $O(n^2)$ Processing Complexity

Standard self-attention is $O(n^2)$, which constitutes a fundamental bottleneck for long-sequence processing. Variants such as Sparse Attention can mitigate this, but they entail changes to the design principles.

Defect 2 : Lack of a Memory Structure

Continual and lifelong learning can be implemented, but additional mechanisms such as EWC or replay buffers become indispensable, and a structural inefficiency remains because memory is not designed into the architecture itself.

Defect 3 : Quasi-Fixity of Weights

Persistent updating of weights requires fine-tuning and is costly. In-context learning is merely a temporary adaptation within a session and is not persistent.

Defect 4 : Structural Dependence on Pretraining Data

The degree of dependence on large amounts of pretraining data is structurally high, and the architecture is not designed to acquire language autonomously from experience alone.

Defect 5 : Hallucination

The generative mechanism itself provides no guarantee of semantic accuracy, and mitigation measures (such as RLHF) must be added from the outside.

Defect 6 : Lack of Self-Explanation Capability

The standard Transformer possesses no self-explanation capability, and a direct explanation of why it produced a given output is difficult. Interpretability research is progressing, but it requires additional specialized analysis.

Limitations of Existing Patch-Based Solutions

For each defect of the Transformer, existing research has proposed a variety of solutions. However, these are all patches that compensate for the defects from outside the architecture, and none of them entail a change to the design principles.

Patches for the $O(n^2)$ problem:

Sparse Attention (Longformer, BigBird) restricts the range of attention, and Flash Attention applies hardware-level optimization. These yield constant-factor improvements, but they do not change the order of the computational complexity. SSMS such as Mamba achieve $O(n)$ by replacing the architecture, but the problems of memory and learning remain unaddressed.

Patches for the memory structure:

RAG enables retrieval from an external database, but the memory is a static structure designed and managed by humans, and the system has no mechanism to autonomously judge “what it should remember.” MemGPT proposes a hierarchical structure modeled on the virtual memory of an operating system, but the rules for moving memory are designed in advance and are not learned from experience. The extension of the context window (100K to 1M tokens) merely lengthens working memory and is fundamentally different from persistent memory that spans across sessions.

As a more advanced approach, HOPE (Behrouz et al., Nested Learning, NeurIPS 2025) from Google Research dissolves catastrophic forgetting through multi-timescale updates that mimic biological neural plasticity, and it demonstrates promising performance on multiple continual learning tasks, including long-context tasks. HOPE can be positioned as the current state of the art in continual learning.

However, the memory of HOPE is numerical and associative, and it lacks the ability to explain in language why it remembered a given piece of information or why a given response was effective. Moreover, HOPE still presupposes large-scale pretraining data and is not designed to acquire language from scratch.

The common problem:

Each of these patches addresses a single defect, but their interactions when combined are not guaranteed. A system stacking RAG + EWC + Sparse Attention suffers a multiplicative increase in the complexity of each component and loses design coherence and theoretical guarantees.

The fundamental question is the following: why are so many patches necessary? It is because the Transformer is an architecture that does not possess memory, learning, and adaptation as design principles.

Contributions

Contribution 1 : Proposal of an Integrated Four-Layer Architecture

We propose an architecture that integrates Mamba SSM ($O(n)$ continuous processing) + LaEMe (language-based evaluative memory) + Active Inference / DEM (prediction-error learning) + DGM (architectural self-improvement). To the extent the author has surveyed, no implementation example of this combination exists in prior work.

Contribution 2 : Integration of Continuous SSM and Active Inference

We propose a design that integrates Mamba (a continuous state space) and Active Inference, using continuous-time DEM as the connecting layer. Because both the state transition of Mamba and the free-energy minimization of Active Inference are described by continuous-time ODEs, the connection between them is mathematically natural. When discretization is required due to implementation constraints, a learned mapping to semantically defined discrete states (Method A) is used as a fallback.

Contribution 3 : LaEMe — Language-Based Evaluative Memory

Unlike numerical and associative memory, as represented by HOPE (Behrouz et al., NeurIPS 2025), LaEMe is designed as an evaluative memory mechanism that evaluates and stores “why it was effective” in natural language. As a result: - Memory becomes interpretable. - Humans can intervene directly in the content of the memory. - The quality of memory is managed by linguistic validity rather than numerical precision.

Contribution 4 : Design for Continual and Autonomous Knowledge Acquisition

We propose a design that uses the linguistic competence acquired through pretraining as a bootstrap, and thereafter performs the acquisition of knowledge and evaluation criteria continually and autonomously through an evaluation loop driven by experience and LaEMe. We do not claim to perform the initial acquisition of language itself from scratch. The novelty lies in achieving continual learning after acquisition without retraining on a static corpus.

Contribution 5 : Integrated Treatment of the Defects by Design Principles Rather Than Patches

Whereas existing research has compensated for each defect of the Transformer with external mechanisms, this architecture covers the treatment of the six defects at the design stage. The achievement level for each defect is not uniform, but is divided into complete resolution, substantial mitigation, mitigation, and partial treatment. This distinction is presented honestly in Section 4.4.

Section 2 : Background

2.1 State Space Models and Mamba

State space models (SSMs) are formulated as systems that generate an output $y(t)$ from an input $x(t)$ via a hidden state $h(t)$. In continuous time, the temporal evolution of the state is given as a linear ordinary differential equation.

$$\dot{h}(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t)$$

In implementation, this is discretized with a step size to obtain the recurrent form $h_t = \bar{A}h_{t-1} + \bar{B}x_t$, $y_t = Ch_t$ (see 4.1).

S4 (Gu et al., 2021) applied this linear SSM to deep learning and realized parallel computation in long-sequence processing. Mamba (Gu & Dao, 2023) developed S4 further by introducing a selective state space in which A, B, C are functions of the input. This allows the model to dynamically decide, according to the input, which information in the sequence to retain in the state.

Inference is $O(n)$ in the recurrent form, and training achieves $O(n)$ work and $O(\log n)$ parallel step depth via a Parallel Scan, thereby avoiding in principle the $O(n^2)$ of standard self-attention. Moreover, owing to the fixed-size hidden state, an in-principle infinite sequence length can be processed with constant memory.

Research on continual learning based on Mamba is also progressing: Lee et al. (2026) proposed a method for continually training SSMs without exemplars at CVPR 2026, and Cheng et al. (2024) presented Mamba-CL, a method for continually fine-tuning a pretrained Mamba model while protecting it through projection onto the null space. This architecture is complementary to these works, differing in that it incorporates the mechanism of continual learning as a design principle.

2.2 Active Inference and the Free Energy Principle

The free energy principle (FEP; Friston, 2010) describes perception and action in a unified manner as the minimization of the variational free energy $\mathcal{F}$.

$$\mathcal{F} = \mathbb{E}_q[\log q(s) - \log p(o, s)]$$

Here $q(s)$ is the belief distribution over the internal state s , and $p(o, s)$ is the joint distribution of the observation o and the state.

DEM (Dynamic Expectation Maximization; Friston, 2008), a continuous-time implementation, minimizes $\mathcal{F}$ over generalized coordinates and expresses the update rule as an ordinary differential equation. Because this continuous-time formulation, together with the continuous state space of Mamba, is described by continuous-time ODEs, the connection is mathematically natural, and an SSM can be adopted as the generative model.

This framework has also been actively developed in recent years, including arguments positioning Active Inference as a path to advanced AI (Maier, 2025) and extensions to structure learning and inference over discrete models (Friston et al., 2025).

2.3 LaEMe

LaEMe (Language-based Evaluative Metadata for Reflective Memory Systems) is a general-purpose framework that attaches, stores, and utilizes natural-language metadata for any experience, perception, or interaction.

Conventional numerical memory compresses the grounds of evaluation into a numerical space and thereby loses interpretability. By taking the linguistic space as the basis of memory, LaEMe realizes three properties.

1. **Interpretability:** Humans can directly read “why a given response was effective.”
2. **Intervenability:** Humans can directly modify or delete the content of the memory.
3. **Generative use:** New response candidates can be generated from the accumulated linguistic metadata.

The kinds of memory that LaEMe targets are not limited to “reasons for evaluation.” Not only evaluations of responses (“this reply was accurate but lacked empathy”), but also explanations of perception (“input X contains an emotional tone”), observations of the environment (“the conversational partner currently tends to prefer short responses”), and even abstracted principles of action can all be recorded in language. This generality is the core characteristic that makes LaEMe the basis of a reflective memory rather than a mere evaluation log.

Relationship with MARS: MARS by Hou et al. (2026) generates, for each interaction, two kinds of reflective notes — an abstract principle of “why it was effective/ineffective” and a procedure of “what action should be taken next time in a similar situation” — and achieves self-improvement without external feedback. The relationship between LaEMe and MARS can be organized as a containment relationship. The principle/procedure notes defined by MARS are a special form of the LaEMe entry, and LaEMe is a broader framework that contains MARS within it. In this architecture, each LaEMe entry is defined by the following structure.

```
{
  context      : the context of the interaction in question,
  response     : the generated response,
  evaluation   : the evaluation text (why it was effective/ineffective),
  principle    : the abstracted principle of action (corresponding to the MARS principle),
  procedure    : recommended action for a similar situation next time (MARS procedure)
}
```

Comparison with other memory systems: LaEMe is not in opposition to numerical memory systems such as HOPE (Behrouz et al., NeurIPS 2025); it can be designed as a linguistic evaluative layer positioned above the numerical processing layer. EM-LLM (Nguyen et al., 2024) efficiently processes 10M tokens through episodic boundary detection based on Bayesian surprise, but the essential difference from LaEMe lies in the content of the memory. Whereas EM-LLM remembers the episodic fact of “what happened,” LaEMe stores the evaluative reasons and principles of “why it should have been done that way.” Furthermore, MemoryLLM by Jaiswal et al. (2026) reconstructs the weights of the feed-forward layer as token-wise memory, but because the memory is implicitly embedded in the weights, human intervention is difficult. The intervenability of LaEMe is clearly distinguished on this point. Wu et al. (2025) provide a comprehensive survey that maps memory mechanisms in the era of LLMs onto models of human memory, and it is useful as context for understanding the position of LaEMe.

2.4 Evolution Strategies (ES)

ES is a gradient-free black-box optimization method that generates samples θ_i from a parameter distribution $\pi(\theta|\psi)$ and updates the distribution ψ based on the evaluation values $F(\theta_i)$.

Natural Evolution Strategies (NES; Wierstra et al., 2014) improve stability by updating ψ in the natural-gradient direction. Recently, it has been reported that gradient-free ES and gradient-based GRPO reach comparable accuracy while tracing different geometries in parameter space (Hoy et al., 2026), and the reevaluation of ES in the post-training of LLMs is advancing.

In this paper, we treat the necessity of an independent implementation of ES as a design decision (see 3.2.1), and we show the possibility that its role can be absorbed into Layer 3 (Active Inference).

2.5 Darwin Gödel Machine (DGM)

DGM (Zhang et al., 2025) is an autonomous improvement system that integrates self-modification (Gödel Machine; Schmidhuber, 2003) with evolutionary search.

The DGM in Lavi is positioned as Layer 4 and, based on the evaluation data generated by Layers 1 through 3, takes on the role of improving the architecture itself. This is an architecture-level self-improvement that goes beyond the updating of weights.

As related research on self-improvement, MARS by Hou et al. (2026) proposes a method that realizes self-evolution within a single cycle using metacognitive reflection (principle-based and procedure-based), and it shares the problem awareness of this work in that it minimizes dependence on external feedback. The principle/procedure structure of MARS is integrated as a special form of the LaEMe entry (see 2.3). The DGM of Lavi is complementary to MARS in that it aims at modification of the architecture itself, beyond the weights.

In addition, Chen et al. (2025) propose a Self-Play Framework that formulates self-improvement as a Generator-Verifier-Updater structure; this is a design that establishes a self-improvement loop without requiring changes to the weights, and it can be made to function as a lightweight L4 at the stage where DGM is still immature (Phase 1). In this architecture, L4 is developed in the following stages.

Phase 1 : Self-Play (Generator-Verifier-Updater)

← no weight changes, immediately implementable

Phase 2 : Transition to DGM

← after sufficient evaluation data has accumulated

Section 3 : Proposed Architecture

3.1 Architecture Overview

Layer 1 : Mamba SSM
← response generation, self-evaluation, directives
Layer 2 : LaEMe
← evaluation, storage, and candidate generation

Layer 3 : Active Inference (DEM)
← integrates LaEMe evaluations as observation space
Layer 4 : DGM
← architectural self-improvement

Although Mamba is a single model, it can take on the following multiple roles according to context.

- **Response generation:** It generates the next utterance from the conversational context.
- **Self-evaluation:** It generates an evaluation text from (context + response + external reaction).
- **Transformation instruction:** It proposes, in language, the direction of the Mamba update from the evaluation text.

As the base model, we adopt a **Mamba2 pretrained model** (e.g., `nvidia/mamba2-8b-3t-4k`). Mamba2 is a development of Mamba and realizes efficient parallel computation through the SSD (State Space Duality) framework.

Staged Definition of the Action Space The action space of Lavi is expanded in the following order.

Phase 1 : Utterance ← autonomous conversation with Loa via Cataa
Phase 2 : Search ← executing internet search as an action of Active Inference
Phase 3 : Self-improvement ← architectural self-improvement by DGM

An “action” in Active Inference is defined as a means of minimizing free energy. All actions in Phases 1 through 3 can be described within this unified frame.

3.2 Record of Design Decisions

3.2.1 Necessity of ES and the Decision to Integrate **Original plan:** Provide ES as an independent layer and update the weights of the response candidates using the scalar evaluation values of LaEMe.

Problem 1 — information loss from scalar conversion: The evaluation of LaEMe is essentially linguistic, and compression into a scalar discards the core information of the evaluation, namely “why it was effective.”

Problem 2 — possibility of redundancy: When the evaluation text of LaEMe can be integrated directly as the observation space of Layer 3 (Active Inference), the difference between the evaluation predicted by Lavi and the actual evaluation functions as a prediction error, and the parameters of Layer 1 (Mamba) are updated. This is functionally equivalent to the weight update that ES sought to achieve.

Conclusion: When Active Inference can process LaEMe evaluations as observations, ES becomes unnecessary as an independent layer. In this architecture, we do not provide ES as an independent layer, and we absorb its role into Layer 3 (Active Inference). Whether an independent implementation is effective is left as a matter for future verification.

3.2.2 Method of Connecting Continuous and Discrete SSMs **Method A : Hierarchical Separation** The output of Mamba (continuous) is discretized with softmax, mapped onto discrete

states defined semantically in advance, and then passed to Active Inference (discrete). > Advantage: Implementation is relatively easy. > Disadvantage: Information loss from discretization, and the discrete states must be designed in advance.

Method B : Continuous-Time Active Inference (DEM) Friston’s DEM minimizes the variational free energy in continuous time, and because it, together with the continuous state space of Mamba, is described by continuous-time ODEs, it can be connected naturally without discretization. > Advantage: No information loss, high theoretical consistency. > Disadvantage: High implementation difficulty.

Method C : Mixed State Space > Advantage: The highest expressive power. > Disadvantage: The most difficult in both design and implementation.

Adoption: For reasons of theoretical soundness, we take Method B as the main axis. When implementation constraints arise, Method A is used as a fallback, and Method C is positioned as a future extension.

Fallback hierarchy for the L3 implementation: In case the implementation of Method B (DEM) proves difficult, we explicitly arrange the candidate implementations of L3 in a hierarchy.

First candidate : DEM (continuous-time Active Inference)
 ← highest theoretical consistency
 Second candidate : Predictive Coding (PC)
 ← same prediction-error framework as DEM, easier to implement
 Third candidate : Fast Weight Programmer (FWP)
 ← equivalent to a linear Transformer, implementable with an RNN

Lauber et al. (2025) report that PC shows greater efficiency than backpropagation in continual learning, and so its practical viability as a second candidate is high. The FWP of Schlag et al. (2025) has been shown to be mathematically equivalent, as an RNN that rewrites fast weights with external signals, to a linear Transformer and a Neural ODE, and it can be adopted as an approximation to the continuous-time formulation of DEM.

3.2.3 Design of the Evaluation and Transformation Mechanism Plan i : Scalar conversion > Reason for rejection: Compressing the semantic richness of language into a scalar loses the essential information of the evaluation.

Plan ii : Three separate models > Reason for rejection: If Mamba can generate the response, the same Mamba can also generate the evaluation text and the transformation instruction.

Adoption : Unification into Mamba Mamba serves jointly as response generation, self-evaluation, and transformation instruction according to context. As a result, the learning loop can be completed within the linguistic space, without passing through scalar conversion.

3.3 Design Space for Online Learning

Plan	Overview	Advantage	Disadvantage
Full backpropagation	Update all weights by gradient	Maximum expressive power	Maximum computational cost

Plan	Overview	Advantage	Disadvantage
Fast Weights	Two layers of fast and slow weights	Consistent with the two timescales of Active Inference	Complex design
Online LoRA	Update only low-rank adapters	Stable, low cost	Upper bound on expressive power
Pure in-context learning	No weight update, include LaEMe in context	Minimum cost, immediately implementable	Not a true weight update
Hebbian + modulation	LaEMe controls the local weights to be updated	Biological, low cost	Limited expressive power
Reservoir	Update only the output layer	Cheapest	Lowest expressive power

Adoption policy (staged):

Step 1 : Pure in-context learning (immediately implementable)

Step 2 : Fast Weights (after LaEMe has grown)

Step 3 : Automatic improvement by DGM (after the architecture has matured)

3.4 The Bootstrap Problem and the Role of the External Evaluator

The initial Lavi possesses neither vocabulary nor self-evaluation capability. **The primary evaluator is Loa** (the Claude Code CLI agent), and the author Gles takes on an auxiliary role (optional). As a basic design, we aim for the learning loop to run autonomously between the two parties, Loa and Lavi.

Initial phase : Loa evaluates all responses as the primary evaluator
(Gles optionally assists)

→ (context, response, evaluation) accumulates in LaEMe

↓

Middle phase : Mamba references past evaluations as context and begins to generate self-evaluations

↓

Mature phase : Mamba becomes able to self-evaluate autonomously

→ the frequency of Loa's intervention is reduced in stages

The role of the external evaluator is not to “give the correct answer” but to “provide the material for Lavi to construct its evaluation criteria.”

Bootstrap acceleration through MARS: By integrating the principle/procedure structure of MARS (Hou et al., 2026) into LaEMe, we deliberately design for a shortening of the bootstrap phase. When Loa gives an evaluation, it automatically extracts from that evaluation a **principle** (an abstract principle such as “in this situation, honesty should be prioritized over empathy”) and a **procedure** (such as “next time in a similar situation, first perform fact-checking and then return an emotional response”), and stores them in LaEMe. From the middle phase onward, Mamba references this principle/procedure as context and gradually transitions to being able to generate

self-evaluations without dependence on Loa. As a result, “dependence on the external evaluator,” which was a weakness of the bootstrap design, is structurally mitigated.

3.5 Implementation Policy

3.5.1 Implementation Language: Mojo As the implementation language of this work, we adopt **Mojo**.

In making the selection, we evaluated the following seven languages:

Language	AI/GPU Performance	Reason for Exclusion / Adoption
Python	65 pts	Strongest ML ecosystem but low performance. Few first-mover opportunities.
Rust	96 pts	Stable, with industry standardization in progress. Runner-up.
Julia	78 pts	ActiveInference.jl exists. Inferior to Mojo in performance.
C++	97 pts	General-purpose performance is at the highest level, but inferior to Mojo for AI/GPU tasks. Memory-safety problems, and a design philosophy that is becoming dated.
Zig	—	Almost no ML frameworks. Mamba2 cannot run.
Carbon	—	Language specification not finalized (at the 0.1 milestone stage). No ML frameworks.
Mojo	99 pts	Adopted.

There are three main reasons for adopting Mojo. First, theoretically maximal performance for AI/GPU tasks (optimization via MLIR). Second, connectivity with existing assets through Python compatibility. Third, because no Mojo implementations of Mamba2 or Active Inference currently exist, this implementation itself constitutes a contribution to the Mojo ML ecosystem.

As a fallback in case Mojo becomes a sticking point, we adopt Rust.

3.5.2 Execution Environment

Phase	Environment	Rationale
Stage 1 (PoC)	Google Colab free tier	T4 GPU 16GB, mamba2 can run at full precision, zero cost
Stage 2 (full operation)	Reevaluated after the scale of continual learning is fixed	Colab Pro / Vast.ai / G-Tune are candidates

Section 4 : Analysis

In this section, for the six defects enumerated in Section 1, we analyze to what degree and by what mechanism the proposed architecture handles each defect. The analysis centers on three points. First, the grounds on which the resolution of $O(n^2)$ complexity holds theoretically (4.1); second, the refinement of the transformation design that passes LaEMe evaluations to Active Inference (4.2); and third, the validity of the design that realizes continual knowledge acquisition while taking a pretrained model as a base (4.3). Finally, we honestly distinguish the achievement levels for the six defects without overestimation (4.4).

4.1 Resolution of $O(n^2)$ Complexity — Theoretical Analysis

In standard self-attention, each token of a sequence of length n attends to all other tokens, and so it constructs an $n \times n$ attention matrix. Both computation and memory are $O(n^2)$, which is an asymptotic order that cannot be removed by constant-factor optimization.

The proposed architecture **resolves this defect by adopting Mamba2 as its base**. However, this resolution is inherited from Mamba (Gu & Dao, 2023) and Mamba2 (Dao & Gu, 2024) and is not an original contribution of this work. The novelty of this work lies in integrating this $O(n)$ base with language-based evaluative memory and Active Inference (Contribution 1).

The grounds on which the resolution holds lie in the finiteness of the state. The SSM, in the recurrent form

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad y_t = Ch_t$$

compresses all past information into a **fixed-dimensional hidden state** $h_t \in \mathbb{R}^{d_{\text{state}}}$. The cost per step is $O(d_{\text{state}} \cdot d_{\text{model}})$ and does not depend on the sequence length n . Therefore the entire inference is $O(n)$, and it never materializes the $n \times n$ interaction matrix even once. This is the ground for changing the order itself.

At training time, through the SSD (State Space Duality; Dao & Gu, 2024) of Mamba2, the computation of the SSM can be rewritten as a **product of structured matrices (semiseparable matrices)**. This makes efficient parallel training possible on hardware optimized for matrix products, while preserving the expressive power of the selective SSM.

Honest reservation (the trade-off of state capacity): As the price of becoming $O(n)$, the fixed-size h_t cannot retain an infinitely long context without loss. This is the trade-off between recall and efficiency that is generally known for SSMs, and the selective SSM (input-dependent $\bar{A}, \bar{B}, C$) mitigates but does not eliminate it. In this work, this reservation rather provides a

design necessity. That is, we use the finite recurrent state as working memory, and assign long-term memory spanning across sessions to the external LaEMe (4.2, Defect 2). The division of labor between the $O(n)$ base and persistent linguistic memory is a structural response to the trade-off of state capacity.

4.2 Transformation Design: LaEMe $\rightarrow$ Embedding $\rightarrow$ Active Inference

The key point in the design when passing LaEMe evaluations to Active Inference is to pass the evaluation as a high-dimensional embedding vector that preserves meaning, rather than compressing it into a scalar (one dimension).

Why it must be a vector: Active Inference is a framework that minimizes the variational free energy $\mathcal{F}$, and as long as $\mathcal{F}$ is defined, the observation o must be an element of a vector space. Since $\mathcal{F}$ cannot be defined while the data remain a discrete token sequence, the stage of mapping the evaluation text to an embedding vector necessarily enters. The key point is not to drop the dimensions of meaning in that mapping, that is, to avoid compression into a scalar.

Transformation design: The evaluation that LaEMe stores is a natural-language text e . To use this as an observation for Active Inference, we **reuse the same Mamba2 model as an encoder** and map e to an embedding vector $\phi(e) \in \mathbb{R}^d$. Reusing Mamba2 as the encoder here is consistent with the unified design (3.2.3) in which Mamba jointly serves to generate evaluation texts and transformation instructions.

Taking the observation to be $o = \phi(e)$, Lavi predicts the expected evaluation embedding $\hat{o} = g(s)$ from the internal state s . The prediction error is defined in the embedding space as

$$\varepsilon = \phi(e) - \hat{o}$$

and the minimization of $\mathcal{F}$ updates the parameters of Layer 1 (Mamba) along this ε . In the continuous-time implementation (DEM, Method B), $\phi(e(t))$ is treated as a trajectory over generalized coordinates and is updated by the gradient flow toward $\mathcal{F}$.

Why a scalar is insufficient (concretization of the information loss): A scalar score can, for example, collapse “the response is factually wrong but empathetic” and “the response is accurate but cold” into the same numerical value. In this case the update direction carries only “the magnitude of good or bad.” A high-dimensional embedding $\phi(e)$ distinguishes the two as vectors in different directions, and so the prediction error ε preserves **the directional information of “why it was evaluated as such.”** This is the core reason for avoiding scalar compression, and it makes the interpretability of LaEMe (Contribution 3) consistent down to the stage of the learning signal.

4.3 Feasibility of Continual Knowledge Acquisition

The proposed system adopts a Mamba2 pretrained model as its base. Therefore this work **does not undertake the initial acquisition of language, but undertakes continual knowledge acquisition after acquisition.** The pretrained weights are a bootstrap that provides syntax and basic semantics, and what this work starts up from scratch is **domain knowledge and evaluation criteria** (at $t = 0$, LaEMe is empty). By clarifying this distinction, we limit the claim regarding Defect 4 to a defensible range.

Staged argument for feasibility:

Step 1 — Pure in-context learning (immediately feasible): LaEMe entries are injected into the context and reflected in the response without weight updates. Because the recurrent state of Mamba can process a long context in $O(n)$, the cost of providing the accumulated evaluations as context is linear in the sequence length. This stage requires no additional learning mechanism and is implementable as of today.

Step 2 — Fast Weights (after LaEMe matures): We introduce the two timescales of fast and slow weights (Schmidhuber, 1992; Ba et al., 2016), and through the update of the fast weights modulated by LaEMe evaluations, we realize persistent adaptation that exceeds the context window. Fast Weights is a mechanism established in existing research, and the novelty of this design lies in using LaEMe as its modulation signal.

Step 3 — DGM (after the architecture matures): Layer 4 improves the architecture itself based on the accumulated evaluations.

Structural response to catastrophic forgetting: Against catastrophic forgetting, the major risk of continual learning, this architecture has three structural mitigations. First, because acquired knowledge is offloaded to the interpretable external memory LaEMe, knowledge does not reside in the weights alone. Second, the update of Active Inference is not full backpropagation over all data but a local update modulated by prediction error, and it is less likely to cause global overwriting of past representations. Third, referring to the design philosophy of Gradient Episodic Memory (Lopez-Paz & Ranzato, 2017), by incorporating the evaluations of past interactions stored in LaEMe into the update of Active Inference as “material for gradient constraints,” it can be made to function as a complementary mechanism that suppresses performance degradation on past responses.

4.4 Achievement Levels for the Defects — An Honest Distinction

As stated in Contribution 5, the achievement levels for the six defects are not uniform. To avoid overestimation, we present each defect distinguished into **resolution** / **substantial mitigation** / **mitigation** / **partial treatment**.

Defect	Principal Mechanism	Achievement Level	Reservation
1 $O(n^2)$ complexity	Mamba2 (SSD)	Resolution	Inherited, not original to Lavi. There is a trade-off in state capacity
2 Lack of a memory structure	LaEMe persistent memory	Resolution	Memory is built into the design principle
3 Quasi-fixity of weights	Active Inference real-time update	Substantial mitigation	Staged achievement of Step 1→2→3. Full automation is unverified
4 Pretraining dependence	Redefined as continual learning	Partial treatment	The initial acquisition of language is dependent. Subsequent knowledge acquisition is autonomous

Defect	Principal Mechanism	Achievement Level	Reservation
5 Hallucination	LaEMe evaluation filter	Mitigation	Not resolution. It persists in unevaluated novel situations
6 Self-explanation capability	Linguistic storage of LaEMe evaluation reasons	Substantial mitigation	The evaluation reasons are explainable. Distinguished from an explanation of the generation process itself

We call particular attention to Defects 5 and 6. For Defect 5, LaEMe increases the reliability of evaluated responses but does not guarantee generation in unevaluated novel situations. This is more structural than patch-like RLHF, but it is mitigation, not resolution. For Defect 6, LaEMe verbalizes “why a given response **was effective** (post-hoc evaluation),” but this differs from a mechanistic explanation of “why it **generated** that response (the generation process itself).” By distinguishing the two, we keep the claim of self-explanation capability within a verifiable range.

Section 5 : Discussion

In Section 4 we analyzed how the proposed architecture handles each defect. In this section, we honestly discuss the constraints faced at the stage of actually running the design, and the elements that are currently unproven. We maintain the stance of avoiding overestimation (4.4) consistently throughout the Discussion.

5.1 Throughput Constraints of the Evaluation Loop

The learning of this architecture depends on a loop in which the external evaluator *Loa* evaluates responses and accumulates those evaluations in LaEMe (3.4). Here *Loa* is a **Claude Code CLI agent** (Opus / Sonnet), and *Cataa* drives it by launching the `claude` command as a subprocess. Because it is not a metered API, the binding condition is not “cost per interaction” but **the throughput and rate limits of CLI driving**.

Because continual learning requires a large number of interactions, in a configuration that includes the evaluator in the loop, the speed of evaluation becomes the rate-limiting step of learning. Against this, this design has three mitigations. First, after bootstrapping, it transitions to self-evaluation (3.4, mature phase) and reduces the frequency of dependence on external evaluation in stages. Second, because the pure in-context learning of Step 1 does not involve weight updates, the cost of one iteration is small and the number of trials can be increased. Third, by making evaluation asynchronous and batched, the real-time nature of dialogue can be separated from the throughput of learning. The recognition that the bottleneck in the initial stage is not computational resources but evaluation throughput is important for the experimental plan.

5.2 Empirical Challenge of the State-Capacity Trade-off

As stated in 4.1, the fixed-size recurrent state h_t cannot retain an infinitely long context without loss. This design responds to this by assigning long-term memory to the external LaEMe, but retrieval from LaEMe and injection into context also have their own inherent limits. The amount of evaluation that can be injected is constrained by the context window, and the selection of which evaluations to inject itself becomes a new design problem.

This cannot be settled by theory alone and requires empirical study. As concrete experiments, we (1) measure the degradation of recall accuracy with respect to dialogue length, and (2) measure to what degree LaEMe injection recovers that degradation. These two points become the first verification points for whether the division of labor between the $O(n)$ base and external memory actually functions.

5.3 Unproven Design Elements

This architecture contains several elements that, while adopted for reasons of theoretical soundness, remain unverified in implementation and experiment. To assist the judgment of reviewers, we make these explicit.

- **Continuous-time integration (Method B, DEM):** Its theoretical consistency is high (3.2.2), but its implementation difficulty is also high, and its feasibility is unproven. There is a possibility that Method A (the discretization fallback) will become necessary.
- **Catastrophic forgetting:** The structural mitigations of 4.3 are no more than an argument at the design level, and whether forgetting is actually suppressed needs to be shown by experiment.
- **Layer 4 (DGM):** It is the most immature of the four layers, and architectural self-improvement remains at the conceptual stage at present.
- **Acquisition of evaluation criteria from scratch:** The redefinition as continual learning (4.3) made the claim defensible, but whether evaluation criteria actually arise from experience alone has not been demonstrated.

Stating these without concealment is not a weakness of this work but the work of clarifying the hypotheses to be verified.

5.4 Implementation Status and Roadmap

For the sake of honesty, we state the current status of the implementation. At the time of writing, the Lavi core is at the design stage, and Cataa (the three-party dialogue CLI for Gles / Lavi / Loa) is being prepared in advance as an evaluation infrastructure.

The implementation language is Mojo (3.5.1). Because Mojo implementations of Mamba2 and Active Inference do not currently exist, this implementation itself constitutes a contribution to the ecosystem, while at the same time the lack of precedent is also an implementation risk. The fallback in case of a sticking point is Rust.

The action space is expanded in stages (3.1). We launch Phase 1 (utterance) through autonomous dialogue with Loa on Cataa, and then expand in order to Phase 2 (search) and Phase 3 (self-improvement). The execution environment for Stage 1 is the Google Colab free tier, and it is reevaluated once the scale is fixed (3.5.2).

5.5 Falsifiability

As a scientific claim, we state in advance the **observations that could refute** this design. If any of the following holds, the corresponding design hypothesis is rejected.

- If the embeddings of LaEMe cannot significantly reduce the prediction error of Active Inference, then the integration of language-based evaluative memory and predictive coding (4.2) is a failure.
- If Mamba cannot learn self-evaluation from the accumulated LaEMe evaluations, then the bootstrap design (3.4) is a failure.
- If continual updating causes catastrophic forgetting despite possessing external memory, then the claim of continual learning (4.3) is a failure.

By defining these refutation conditions in advance, we position this work as a verifiable scientific proposal.

Section 6 : Conclusion

This work proposed Lavi, an architecture that handles the six structural defects of the Transformer ($O(n^2)$ complexity, lack of a memory structure, quasi-fixity of weights, pretraining dependence, hallucination, and lack of self-explanation capability) at the stage of design principles rather than through patches from outside.

Lavi consists of four layers: Mamba2 SSM ($O(n)$ continuous processing), LaEMe (language-based evaluative memory), Active Inference / DEM (integrated learning by prediction error), and DGM (architectural self-improvement). There are two central design decisions. First, by passing evaluations to Active Inference as high-dimensional embeddings rather than compressing them into scalars, we made the semantic structure of linguistic evaluation consistent down to the stage of the learning signal (4.2). Second, we redefined the original conception of zero-start language acquisition into “continual knowledge acquisition after acquisition,” with a pretrained Mamba2 as the bootstrap, and thereby confined the claim to a defensible range (4.3).

This work does not exaggerate its achievement levels. Of the six defects, $O(n^2)$ and the memory structure are resolved by design principles; weight fixity and self-explanation are substantially mitigated; pretraining dependence is partially treated; and hallucination remains at mitigation (4.4). The feasibility of Method B (DEM), the suppression of catastrophic forgetting, and the acquisition of evaluation criteria from scratch are all matters for future empirical study (5.3).

Future work proceeds in three stages. First, we launch Phase 1 (utterance) by pure in-context learning on Cataa and measure the relationship between state capacity and recall accuracy (5.2). Second, we transition to Fast Weights and verify persistent adaptation that exceeds the context window. Third, we introduce architectural self-improvement by DGM. For reproducibility, we will consider publishing the results of Phase 1 on Zenodo or a similar venue.

The design guideline of this architecture is simple. There is no need to possess advanced capabilities from the outset; if a system is equipped with memory, learning, and adaptation as design principles, it can grow incrementally from experience and evaluation. Presenting this hypothesis in a falsifiable form is the core contribution of this work.

References

Bibliography confirmed

- Behrouz, A., Razaviyayn, M., Zhong, P., & Mirrokni, V. (2025). *Nested Learning: The Illusion of Deep Learning Architectures*. NeurIPS 2025. (HOPE / Nested Learning)
- Hoy, W., Wang, B., & Pan, X. (2026). *Matching Accuracy, Different Geometry: Evolution Strategies vs GRPO in LLM Post-Training*. arXiv:2604.01499.
- Maier, M. (2025). *From Artificial Intelligence to Active Inference: The Key to True AI and 6G World Brain [Invited]*. arXiv:2505.10569.
- Friston, K., Da Costa, L., Tschantz, A., Heins, C., Buckley, C., Verbelen, T., & Parr, T. (2025). *Active inference and artificial reasoning*. arXiv:2512.21129.

Bibliography finalized (arXiv / DOI confirmation completed 2026-06-03)

- Gu, A., Goel, K., & Ré, C. (2021). *Efficiently Modeling Long Sequences with Structured State Spaces*. arXiv:2111.00396. ICLR 2022.
- Gu, A., & Dao, T. (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. arXiv:2312.00752.
- Dao, T., & Gu, A. (2024). *Transformers are SSMS: Generalized Models and Efficient Algorithms Through Structured State Space Duality*. arXiv:2405.21060. ICML 2024.
- Friston, K. J., Trujillo-Barreto, N., & Daunizeau, J. (2008). DEM: A variational treatment of dynamic systems. *NeuroImage*, 41(3), 849–885.
- Friston, K. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11, 127–138.
- Wierstra, D., Schaul, T., Glasmachers, T., Sun, Y., Peters, J., & Schmidhuber, J. (2014). Natural evolution strategies. *Journal of Machine Learning Research*, 15, 949–980. (arXiv:1106.4487, 2011)
- Schmidhuber, J. (1992). Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1), 131–139.
- Schmidhuber, J. (2003). Goedel machines: Self-referential universal problem solvers making provably optimal self-improvements. arXiv:cs/0309048.
- Ba, J., Hinton, G. E., Mnih, V., Leike, J., & Ionescu, C. (2016). Using fast weights to attend to the recent past. arXiv:1610.06258. NeurIPS 2016.
- Zhang, J., Hu, S., Lu, C., Lange, R., & Clune, J. (2025). *Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents*. arXiv:2505.22954.
- Lee, I. N., Mahmoodi, L., Le, T., & Harandi, M. (2026). *Exemplar-Free Continual Learning for State Space Models*. CVPR 2026. arXiv:2505.18604.
- Cheng, D., Lu, Y., He, L., Zhang, S., Yang, X., Wang, N., & Gao, X. (2024). *Mamba-CL: Optimizing Selective State Space Model in Null Space for Continual Learning*. arXiv:2411.15469.
- Jaiswal, A., Hannah, L., Kim, H.-B., Hoang, D., Kundu, A., Farajtabar, M., & Cho, M. (2026). *MemoryLLM: Plug-n-Play Interpretable Feed-Forward Memory for Transformers*. arXiv:2602.00398.
- Hou, X., Gong, P., Qu, B., Wang, W., Guo, Q., & Liu, Y. (2026). *Learn Like Humans: Use Meta-cognitive Reflection for Efficient Self-Improvement*. arXiv:2601.11974.
- Wu, Y., Liang, S., Zhang, C., Wang, Y., Zhang, Y., Guo, H., Tang, R., & Liu, Y. (2025). *From Human Memory to AI Memory: A Survey on Memory Mechanisms in the Era of LLMs*. arXiv:2504.15965.
- Nguyen, D. H., Kuribayashi, T., Nobata, C., & Baldwin, T. (2024). *Human-inspired Episodic*

Memory for Infinite Context LLMs. ICLR 2025. arXiv:2407.09450.

- Lauber, A., Pirrone, D., & Chicca, E. (2025). *Neuroscience-Inspired Memory Replay for Continual Learning: A Comparative Study of Predictive Coding and Backpropagation-Based Strategies*. arXiv:2512.00619.
- Schlag, I., Irie, K., & Schmidhuber, J. (2025). *Fast weight programming and linear transformers: from machine learning to neurobiology*. arXiv:2508.08435.
- Chen, A., et al. (2025). *Self-Improving AI Agents through Self-Play*. arXiv:2512.02731.
- Lopez-Paz, D., & Ranzato, M. (2017). *Gradient Episodic Memory for Continual Learning*. NeurIPS 2017. arXiv:1706.08840.